AI BlogNest API

Project Documentation

Team Members

Gayathri.B

Gayathri.R

Harini.K

Gracy.R

Gifty Alsida

ContentGen - Generate high-quality blog content using AI

Description

The ContentGen backend is a powerful, secure, and intelligent REST API platform designed to simplify blog creation, content management, and AI-assisted writing. Built with Node.js and Express.js, the application uses MongoDB as its primary database with Mongoose ODM for efficient data modeling, validation, and seamless database interactions.

The platform integrates Google Gemini AI to provide intelligent features such as AI-powered blog content generation, AI FAQ assistance, AI WeatherWise insights, and AI FitTrack recommendations, enabling users to create high-quality, context-aware content with minimal effort.

Security is a core aspect of the architecture, implementing JWT (JSON Web Tokens) for stateless authentication, role-based access control (Admin, Editor, Author, Reader) for authorization, and bcrypt.js for secure password hashing. The backend also supports comprehensive blog management, categories and tags, comment management, media metadata handling, advanced search capabilities, and centralized error handling. Additional security mechanisms such as request validation, input sanitization, rate limiting, and structured logging ensure the platform remains reliable, scalable, and resilient against common web vulnerabilities.

Designed with a modular and scalable architecture, ContentGen serves as a modern AI-powered Content Management System (CMS) that combines traditional blogging capabilities with Generative AI to deliver a smarter, faster, and more personalized content creation experience.

Scenario-Based Case Study

Background

Priya is a content creator who manages multiple blogs but struggles to organize posts, schedule publications, manage categories/tags, and track reader engagement efficiently.

Problem

- Difficulty managing multiple drafts, categories, and tags.
- Limited visibility into post performance and engagement analytics.
- Time-consuming processes for content approval and comment moderation.
- No streamlined system for handling media uploads or metadata.

- Lack of collaborative permission tiers for editors, authors, and administrators.

Solution

The BlogNest backend delivers a centralized, secure API wrapper that decouples application business logic from frontend delivery systems. It introduces an internal permission matrix controlling specialized services for content scheduling, category taxonomy, media path registration, and granular user roles.

Usage

- Author Usage: Creates blog posts, uploads media pathways, organizes content drafts, and reviews their distinct post-performance data.
- Editor/Admin Usage: Accesses protected endpoints to review content queues, change publishing statuses, moderate visitor comments, and orchestrate user privilege upgrades.
- Reader Usage: Queries public list routes to consume posts and invokes comment creation endpoints.

Outcome

By offloading workflow controls to a robust backend, content execution delays are minimized. Data structural integrity is strictly maintained through schema validation, security filters guarantee that only authorized roles perform administrative shifts, and persistent tracking provides raw analytics to measure post optimization metrics accurately.

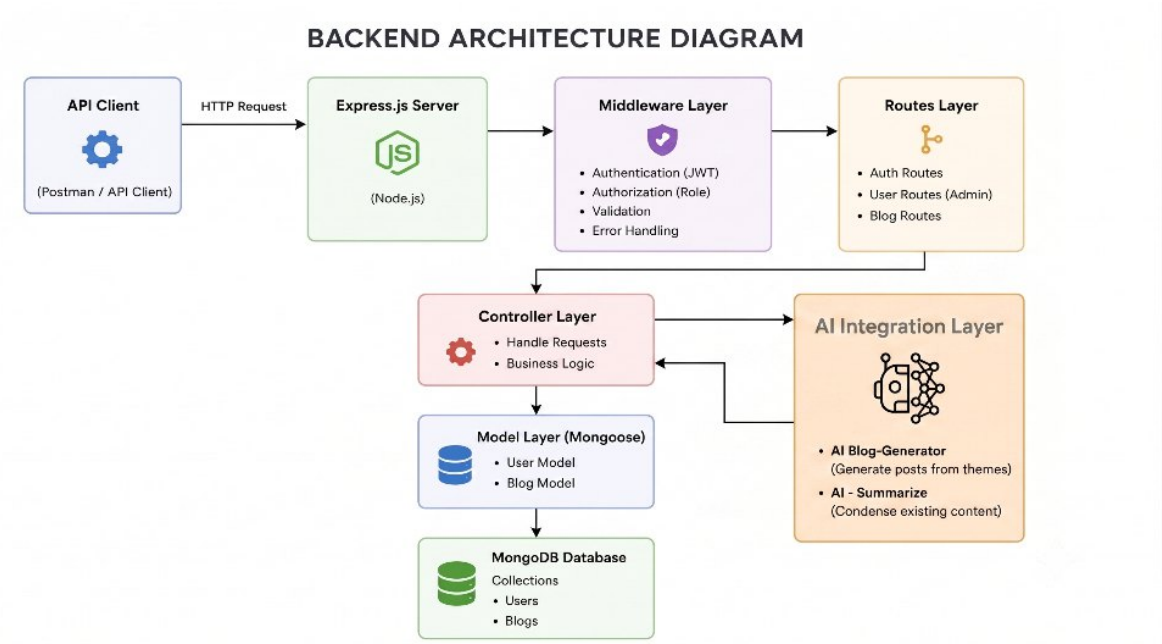
1. Software Requirements

- Operating System: Windows 10/11, macOS, or Linux (supports cross-platform operations).
- Node.js (v16 or above): Runtime ecosystem running server logic and routing infrastructure.
- npm (v8 or above): Node package manager to control operational dependencies.
- Express.js: Lightweight routing web framework to construct backend RESTful entry points.
- MongoDB: Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- Postman: API verification toolkit to assert schema validation responses across administrative route shields.
- Code Editor: Visual Studio Code or similar IDE environment.

2. Hardware Requirements

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- Storage: 1 GB of available disk workspace.

Epic:-1 PROJECT ARCHITECTURE



The BlogNest application exposes structured endpoints handled by a layered server architecture model. Incoming requests undergo filter processing before interacting with data services.

Components Definition

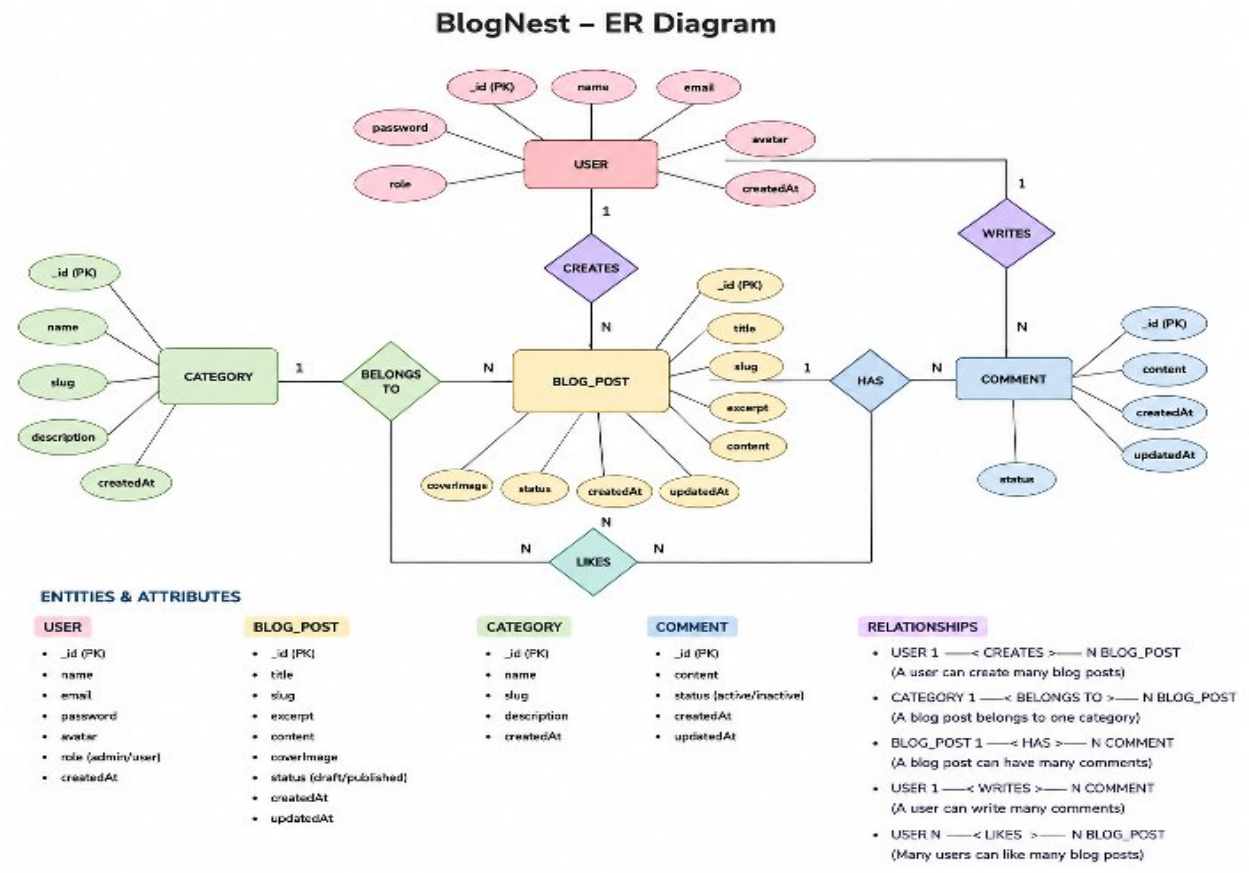
- Interface: API tools like Postman query distinct route paths to extract JSON entities.
- Express Server Gateway: Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers.
- Authentication & Role Matrix Middleware: Extracts tokens from headers, validates user context signatures, and evaluates if the specific role satisfies authorization thresholds for guarded resource paths.
- Granular Services (Logic Modules): Enforces isolated domain rules for blog lifecycles (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators.
- Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.
- AI Flow (Service Integration): Provides intelligent content automation, including automated blog post generation and real-time content summarization. This layer connects to external AI models via specialized endpoints (/api/ai/generate-blog and /api/ai/summarize), allowing

the controller layer to offload complex text processing tasks and enhance content creation workflows.

- BlogGenerator with AI: A specialized service that automates the generation of blog content, which integrates with the AI Flow service to streamline the drafting process.

Entity-Relationship (ER) Diagram Description

The underlying document structure manages several collections bound by logical application references:



1. User Entity

- _id (Primary Key - ObjectId)

- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Post Entity

- _id (Primary Key - ObjectId)
- userID (Foreign Key referencing User Collection)
- name (String, Required - represents Title)
- photo (Object/String pathway link)
- message (String - Core content body)
- likes (Number, Default: 0)
- timestamps (True)

Key Structural Mappings

- User to Post (One-to-Many): An author can submit multiple blog documents.
- User to Comment (One-to-Many): Readers/Users can append multiple response comments across independent post nodes.
- Post to Comment (One-to-Many): A single blog document stores individual referenced comment submissions.

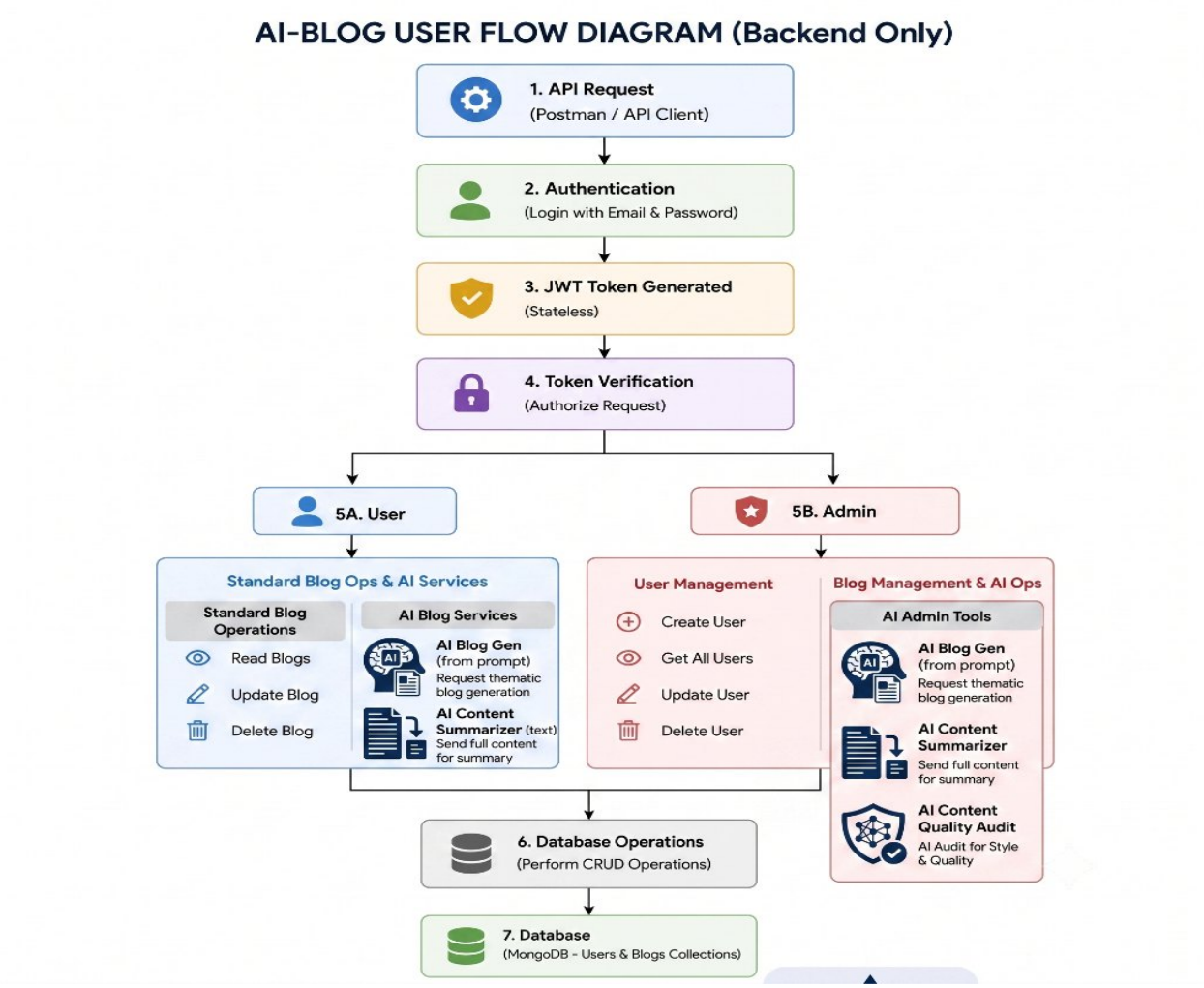
Key Features

- Role-Based Route Guarding: Strict token processing blocks unauthorized tiers from modifying core settings.
- CRUD Blog Execution Engine: Complete schema management over content definitions, status states, and asset assignment paths.
- Categorization System: Native tag indices facilitate filtering and querying for enhanced discoverability.
- Comment Moderation Logic: Protected entry points for updating moderation flags (approved, pending, spam).
- Sanitization Safeguards: Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.
- I-Driven Blog Generation Pipeline: Integrates external Large Language Model (LLM) abstractions and context-aware prompt templates to synthesize full-length, structured blog posts from high-level thematic inputs.
- Intelligent Content Summarization Engine: Leverages semantic compression layers to parse extensive article payloads, dynamically generating concise abstracts and automated metadata summaries.

Roles and Responsibilities

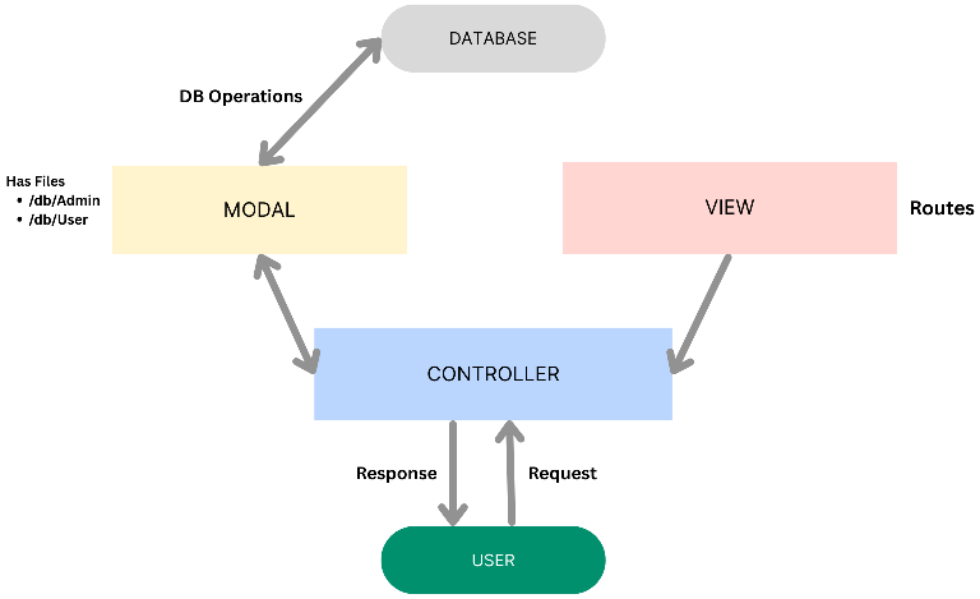
- Admin: Complete environment override control. Can manage user database registrations, alter role assignments, prune malicious comments, and inspect full logging entries.
- Editor: Focuses on editorial workflows. Reviews pending draft posts, marks records for instant publication, and handles category cleanup scripts.
- Author: Restricted account access. Can issue creation statements only for posts bound to their explicit account record.
- Reader: Consumer access profile. Reads published materials and posts comments under an ongoing authenticated identity.

User Flow:



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

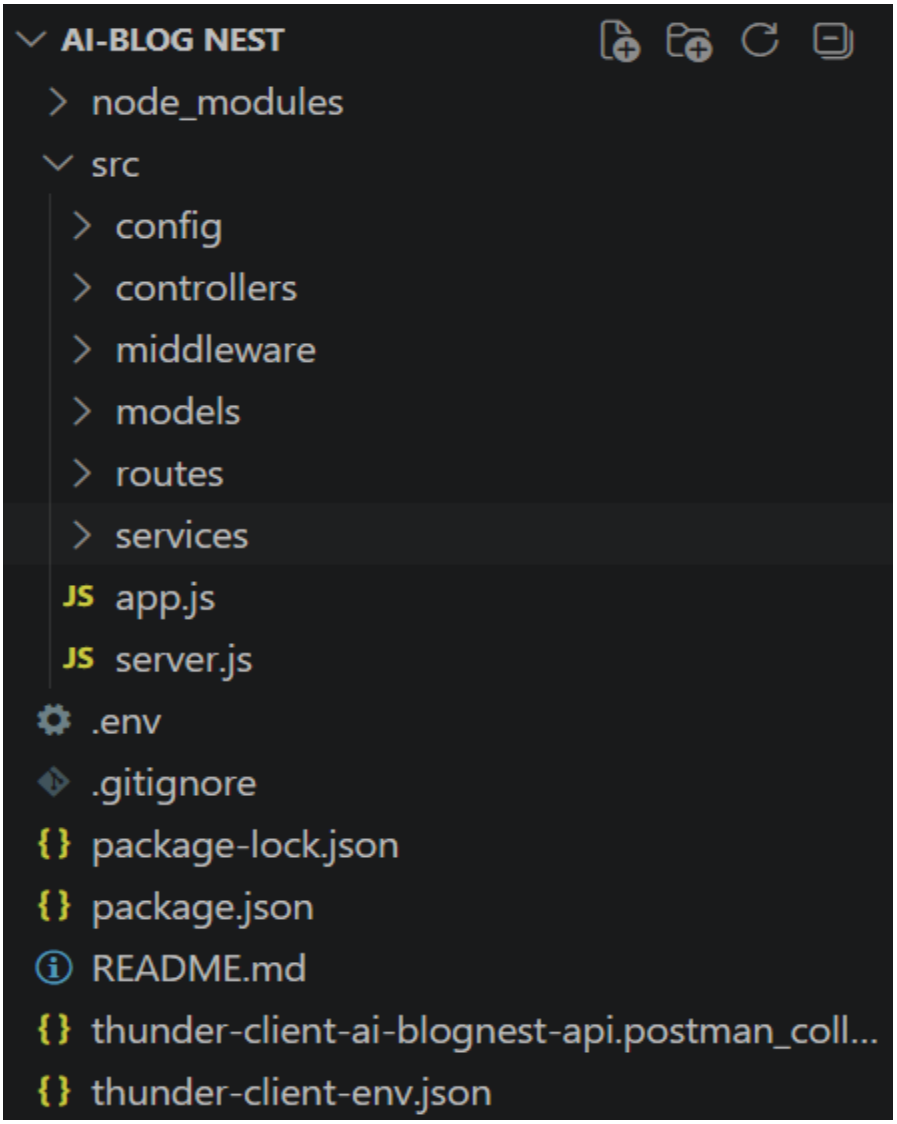
In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.

Epic2: Project Setup and Configuration

Creating project folder

1. Create a new folder with your <project name>.
2. Inside that folder create one new folders.
3. Name another one as Server.

4. Now open that folder in VS Code.



Server setup (npm init)

- Open Server folder in terminal of VScode.
  - npm init -y
- Create files:
  - server.js
- Create folders:
  - models
  - controllers
  - routes

Epic 3: Backend Development

Server.js



App.js

app.js

src > app.js > ...

```
const express = require('express');
const cors = require('cors');
const morgan = require('morgan');
const authRoutes = require('./routes/authRoutes');
const blogRoutes = require('./routes/blogRoutes');
const aiRoutes = require('./routes/aiRoutes');
const errorMiddleware = require('./middleware/errorMiddleware');

const app = express();

app.use(cors());
app.use(express.json());
app.use(morgan('dev'));

app.use('/api/auth', authRoutes);
app.use('/api/blogs', blogRoutes);
app.use('/api/ai', aiRoutes);

app.use(errorMiddleware);

app.get('/', (req, res) => {
  res.json({ message: 'AI BlogNest API is running' });
});

module.exports = app;

```

Epic4: Database Configuration

DB Connection:-



1. Configure MongoDB:

- Install Mongoose.
  In your Node.js project directory, run:
  npm install mongoose
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

1. Identity Resource Document Schema (schemas/userModel.js)

Enforces field criteria validations on incoming database creations:

User.js

src > models > User.js > ...

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: 6,
    },
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model('User', userSchema);

```

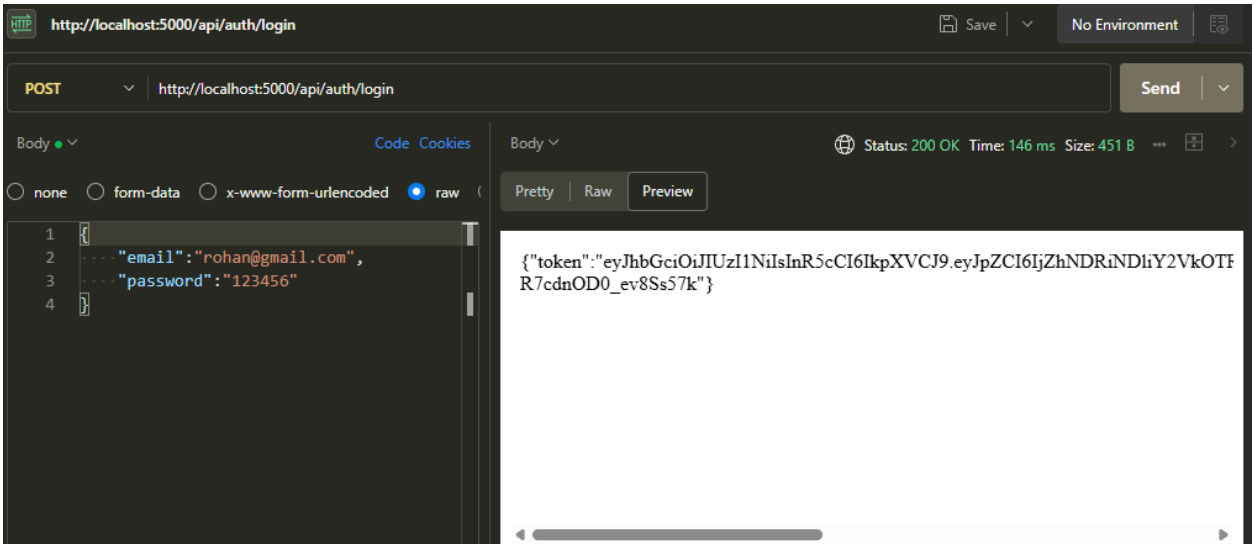
Epic 5: Project Executions:

- Install Postman or ThunderClient (VS Code extension).

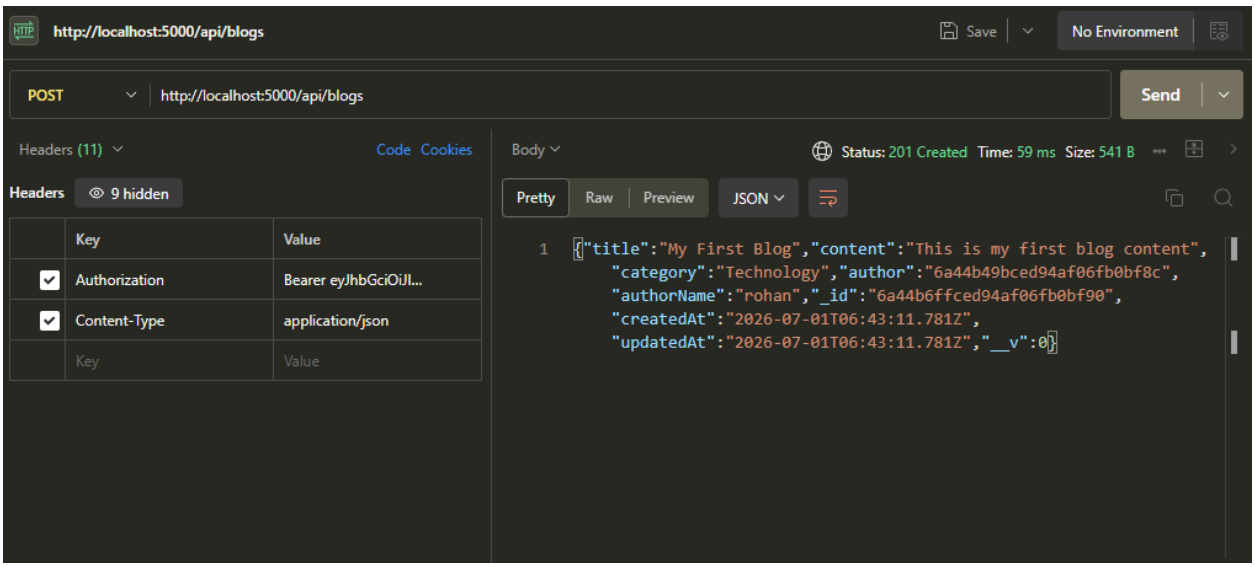
- Start the backend server:
  “npm start”

API Testing

User Registration :1
User Login:

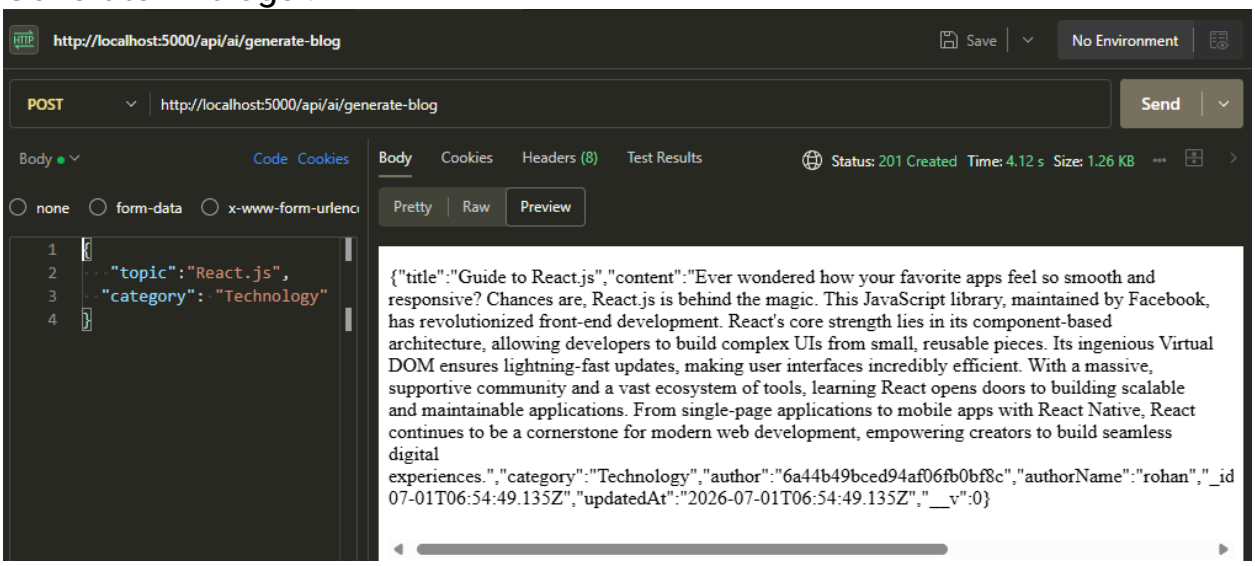


Create Blogs :

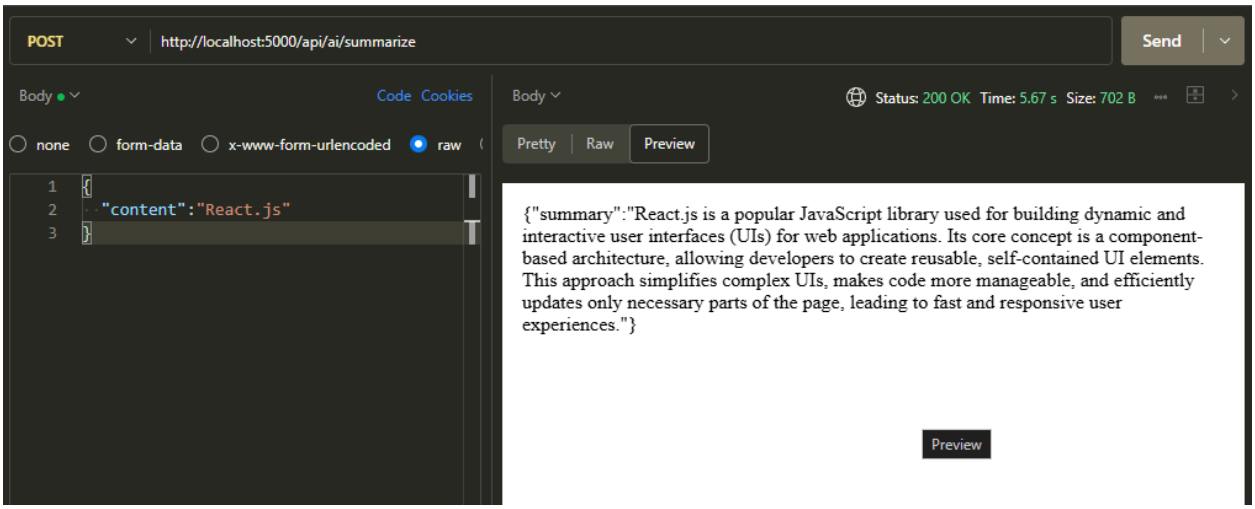


Fetch Blogs : 1

Generate AI blogs :



Summarize Blogs :



Demo Video :Demo Videos

API Testing : API testing Video

CodeDrive:Code